pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Case Study: Convexity and the September Book

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem definition: the line's miss, and a book rebuilt

The Desk Lead's Tenth Note

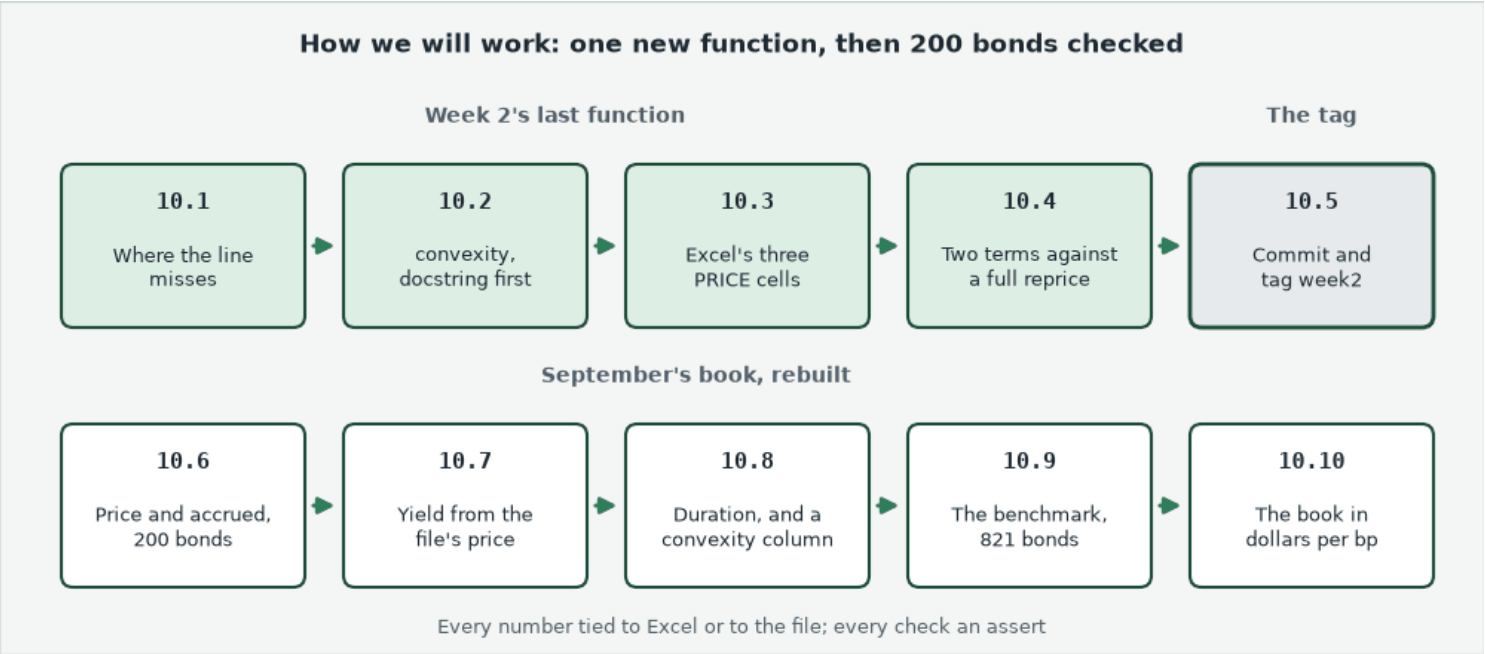
Notebook 09 put our four positions at 6,154.97 dollars a basis point: a straight line. For a 100 basis point move, how far off is that line, and what measures the bend? Put that measure in "bondmath", tied to Excel, which has no function for it. Then September's holdings: 200 bonds priced by someone else's program. Rebuild every bond's price, accrued, yield and duration before we use the file, and run the same checks on the benchmark it came from. Last, what does the book carry in dollars per basis point, and what is its duration? Describe the book: no view on any bond.

You are the desk's analyst: answer the five asks with your own module, and tie every number to Excel or to the file. Every check is an "assert", so one bond that disagrees stops the notebook.

Notebook 09's four positions, settling Monday 30 November 2026, typed in 10.1. "holdings.csv", the September book: 200 bonds as of 30 September 2026, made by a program, the course's data generator. Issuers and CUSIPs are invented; every statement describes the data.

Column	Meaning	Used for
as_of_date	2026-09-30 for every bond, also the settlement date	input
coupon, maturity, yield_pct	coupon and yield in percent, the maturity date	input
price, accrued, duration	clean price and accrued per 100, modified duration in years, 3 decimals	checked
par_held, market_value	face held and market value, in dollars	the book's totals

How we will work



One new function, then 200 bonds checked with every function of the week.

```
...   [100%]
27 passed in 0.50s
```

10.1 Duration's straight line, and 10.2 the bend

Duration's estimate is a straight line in the yield: change in dirty price = -modified duration x dy x dirty price, with dy the yield change as a decimal (100 bp is 0.01).

```
halvering_maturity = date(2036, 9, 30)
dirty_now = bondmath.dirty_price(settlement, halvering_maturity, 7.5, 7.7)
modified = bondmath.modified_duration(settlement, halvering_maturity, 7.5, 7.7)
print(f'Dirty price {dirty_now:.6f} per 100, modified duration {modified:.6f} years')
print(f'The line for +100 bp: {-modified * 0.01 * dirty_now:.4f} per 100')
Dirty price 99.872491 per 100, modified duration 6.759766 years
The line for +100 bp: -6.7511 per 100
```

Duration is the slope of the price-yield curve over the price; convexity is how fast that slope changes, the curvature, also over the price. It is a sum over the cash flows, with five pieces: each cash flow CF, its time t = (w + k) / f in years as in notebook 09, the frequency f, the yield y as a decimal, and the dirty price.

convexity = sum of CF x t x (t + 1/f) / (1 + y/f) ** (f x t + 2), divided by the dirty price

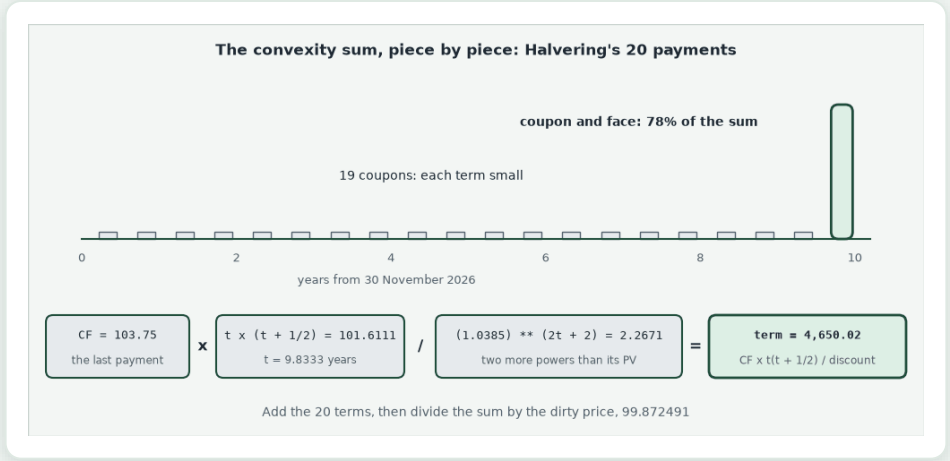
With it, the estimate gains a second term:

change in dirty price / dirty price = -modified duration x dy + 0.5 x convexity x dy ** 2

The second term is a square, positive whether yields rise or fall: the sign of 10.1's miss. Time enters about squared, so before discounting, a payment twice as far away adds about four times as much.

10.2 convexity, docstring first

```
%%writefile -a bondmath.py
def convexity(settlement: date, maturity: date, coupon_pct: float, yield_pct: float, frequency: int = 2,
              basis: str = "30/360") -> float:
    dates = coupon_dates(settlement, maturity, frequency)
    _, days_in_period, days_to_next = coupon_period_days(settlement, maturity, frequency, basis)
    coupons_left = len(dates) - 1
    w = days_to_next / days_in_period
    period_yield = yield_pct / 100 / frequency
    coupon = coupon_pct / frequency
    total_pv, curvature = 0.0, 0.0
    for k in range(coupons_left):
        cash_flow = coupon + 100 if k == coupons_left - 1 else coupon
        time_years = (w + k) / frequency
        total_pv += cash_flow / (1 + period_yield) ** (w + k)
        curvature += (cash_flow * time_years * (time_years + 1 / frequency)
                      / (1 + period_yield) ** (frequency * time_years + 2))
    return curvature / total_pv
```



The docstring states the units (years squared, unscaled), the formula, the estimate it feeds, and Excel's three-cell check. The loop is notebook 09's "macaulay_duration" with t x (t + 1/f) in place of t, and two more powers of (1 + y/f).

Some sources divide convexity by 100; some halve it and drop the estimate's 0.5. This course's is unscaled: ask which scaling any other number uses before comparing.

10.3 When Excel has no convexity function

Excel has "DURATION" and "MDURATION", but no convexity function. So the check comes from the definition: notebook 09's bump took one slope from two prices; three prices 1 bp apart give two slopes, and the bend is how much they differ.

central difference = (P(y + h) + P(y - h) - 2 x P(y)) / (dirty price x h ** 2), with h = 0.0001, 1 bp as a decimal

Clean prices will do: accrued does not move with the yield, so it cancels.

```
h = 0.0001
p_mid = bondmath.price(settlement, halvering_maturity, 7.5, 7.7)
p_down = bondmath.price(settlement, halvering_maturity, 7.5, 7.7 - 0.01)
p_up = bondmath.price(settlement, halvering_maturity, 7.5, 7.7 + 0.01)
central = (p_up + p_down - 2 * p_mid) / (dirty_now * h ** 2)
print(f'Central difference {central:.6f}, convexity {halvering_convexity:.6f}')
print(f'Gap {abs(central - halvering_convexity):.1e}')
```

It measures the average curvature across the 2 bp step, not at one point, so the tolerance is 1e-3.

Excel side by side. Halvering in a blank sheet.

Job	Excel
Clean price, A1	=PRICE(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100,100,2,0)
1 bp lower, A2	=PRICE(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100-0.0001,100,2,0)
1 bp higher, A3	=PRICE(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100+0.0001,100,2,0)
Accrued, A4	=100*7.5/100/2*COUPDAYBS(DATE(2026,11,30),DATE(2036,9,30),2,0)/COUPDAYS(DATE(2026,11,30),DATE(2036,9,30),2,0)
Central difference, A5	=(A2+A3-2*A1)/((A1+A4)*0.0001^2)

The test, and 10.4 two terms against a full reprice

The test holds "convexity" to "convexity_fd" on every reference row outside the final coupon period, where "PRICE" is simple interest (notebook 08's 8.5). The comprehension's "if" keeps those rows, as in notebook 09's bump test. The file's 200 holdings rows are among them, so the test also ties "convexity" to today's book.

```
%%writefile -a test_bondmath.py
def test_convexity_matches_excel_finite_difference():
    rows = [row for row in REFERENCE if row["coupnum"] != "1"]
    for row in rows:
        result = bondmath.convexity(*bond_inputs(row))
        assert result == pytest.approx(float(row["convexity_fd"]), abs=1e-3), row["case_id"]
```

```
...   [100%]
28 passed in 0.50s
```

"convexity" matches Excel's three "PRICE" cells on all 294 rows outside the final period.

10.4 Four Bonds at +100 bp

```
bonds['dirty'] = [bondmath.dirty_price(settlement, m, c, y)
                  for m, c, y in zip(maturities, coupons, yields)]
bonds['dirty_up'] = [bondmath.dirty_price(settlement, m, c, y + 1)
                     for m, c, y in zip(maturities, coupons, yields)]
bonds['estimate_pct'] = (-bonds['modified'] * 0.01 + 0.5 * bonds['convexity'] * 0.01 ** 2) * 100
bonds['full_pct'] = (bonds['dirty_up'] / bonds['dirty'] - 1) * 100
bonds['miss_pct'] = bonds['full_pct'] - bonds['estimate_pct']
bonds[['issuer', 'estimate_pct', 'full_pct', 'miss_pct']].round(6)
```

```
Estimate for +100 bp, percent, A6
=(-MDURATION(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100,2,
0)*0.01+0.5*A5*0.01^2)*100
Full reprice for +100 bp, percent, A8
=(PRICE(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100+0.01,100,2,
0)-A1)/(A1+A4)*100
```

10.5 Commit, and tag the week

"convexity" is week 2's last function: commit it, and name the commit that ends the week.

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
subject = 'Add convexity, checked against a central difference on Excel PRICE'
body = "Duration's straight line misses the bend in the price-yield curve; convexity measures it."
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
 M bondmath.py
 M test_bondmath.py
 bondmath.py    | 30 ++++++++++++++++++++++++++++++
 test_bondmath.py |  9 +++++++++
 2 files changed, 39 insertions(+)
```

```
!git -C "{work}" tag week2
!git -C "{work}" log --oneline --decorate
646a02a (HEAD -> main, tag: week2) Add convexity, checked against a central difference on Excel PRICE
5fcb7c6 Notebook 09: macaulay_duration, modified_duration, and dv01
20b3546 Notebook 08: price, dirty_price, and yield_to_maturity
98124d3 Notebook 07: days_30_360, coupon_period_days, and accrued_interest
9fe8f04 Notebook 06: add_months, coupon_dates, and a .gitignore
322960d (tag: week1) Notebook 05: tests of par pricing and of price against yield
7147a11 Notebook 04: yield_from_price
3f57cac Notebook 03: convert_yield
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

10.6 The book's price and accrued, rebuilt

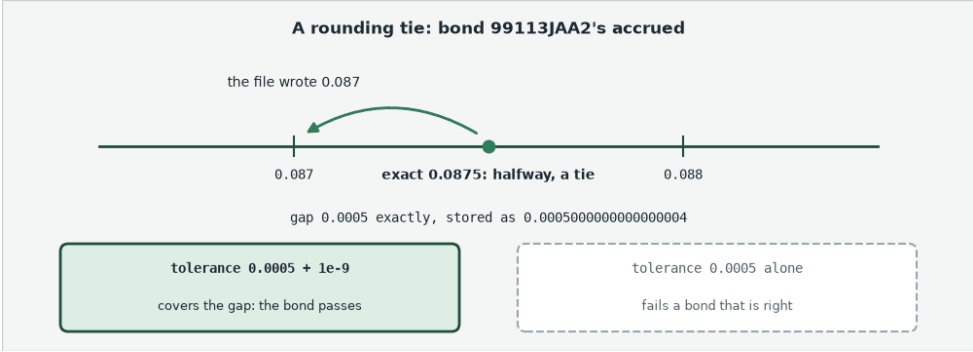
September's holdings were priced by someone else's program and rounded to 3 decimals: rebuild each number before trusting it.

```
book = pd.read_csv('holdings.csv')
book['settle'] = pd.to_datetime(book['as_of_date']).dt.date
book['maturity_date'] = pd.to_datetime(book['maturity']).dt.date
assert len(book) == 200 and book['cusip'].is_unique
assert book['as_of_date'].nunique() == 1 and (book['maturity_date'] > book['settle']).all()
print(len(book), 'bonds as of', book['settle'].iloc[0])
200 bonds as of 2026-09-30
```

A column rounded to 3 decimals can sit up to 0.0005 from the exact number. A yield solved from a rounded price carries that rounding with it.

```
ROUNDED_3 = 0.0005 + 1e-9
YIELD_FROM_ROUNDED_PRICE = 0.001
```

```
inputs = list(zip(book['settle'], book['maturity_date'], book['coupon'], book['yield_pct']))
book['price_calc'] = [bondmath.price(*bond) for bond in inputs]
book['accrued_calc'] = [bondmath.accrued_interest(s, m, c) for s, m, c, y in inputs]
price_gap = (book['price_calc'] - book['price']).abs()
accrued_gap = (book['accrued_calc'] - book['accrued']).abs()
assert price_gap.max() <= ROUNDED_3 and accrued_gap.max() <= ROUNDED_3
print(f'Largest price gap {price_gap.max():.7f}, largest accrued gap {accrued_gap.max()}')
Largest price gap 0.0004985, largest accrued gap 0.0005000000000000004
```



Both pass on all 200 bonds, but the accrued gap sits a hair above 0.0005.

The exact accrued ends in a 5 at the fourth decimal; the file rounded it down. Floating point stores neither number exactly, so the gap of exactly 0.0005 comes out a hair above it.

Price and accrued tie on all 200 bonds; the one gap a hair above 0.0005 is a rounding tie.

10.7 Yields from the file, 10.8 duration and convexity

Now the other way: solve each yield from the file's rounded price with notebook 08's "yield_to_maturity".

```
book['yield_calc'] = [bondmath.yield_to_maturity(s, m, c, p) for s, m, c, p
                      in zip(book['settle'], book['maturity_date'], book['coupon'], book['price'])]
book['duration_calc'] = [bondmath.modified_duration(*bond) for bond in inputs]
book['yield_gap'] = (book['yield_calc'] - book['yield_pct']).abs()
assert book['yield_gap'].max() <= YIELD_FROM_ROUNDED_PRICE
book.nlargest(5, 'yield_gap')[['cusip', 'duration_calc', 'price', 'yield_gap']].round(6)
```

10.8 Duration Against the File, and Convexity

```
duration_gap = (book['duration_calc'] - book['duration']).abs()
assert duration_gap.max() <= ROUNDED_3
print(f'Largest duration gap {duration_gap.max():.7f} years')
book['convexity'] = [bondmath.convexity(*bond) for bond in inputs]
book[['duration_calc', 'convexity']].describe().round(4)
Largest duration gap 0.0004982 years
```

	duration_calc	convexity
count	200.0000	200.0000
mean	6.4704	72.5274
std	3.3505	76.3239
min	0.9271	1.3480
25%	3.8800	17.9547
50%	6.3062	50.0327
75%	8.1401	81.9089
max	16.0425	369.3686

The longest duration is 17 times the shortest; the largest convexity, 274 times the smallest.

```
excel_fd = reference[reference['group'] == 'holdings'].set_index('case_id')['convexity_fd']
convexity_gap = (book.set_index('cusip')['convexity'] - excel_fd).abs()
assert len(convexity_gap) == 200 and convexity_gap.max() < 1e-3
print(f'Largest gap {convexity_gap.max():.1e}, on {convexity_gap.idxmax()}')
Largest gap 2.2e-04, on 99053BAB9
```

All four columns rebuilt, and a fifth added that Excel confirms: the note's third ask.

10.9 Two systems disagree, again: the benchmark

"benchmark.csv", the wider file the holdings came from, has 821 bonds in the same columns. Rather than copy four cells, put the checks in one function for any file of this shape.

```
def column_gaps(file):
    settle = pd.to_datetime(file['as_of_date']).dt.date
    maturity = pd.to_datetime(file['maturity']).dt.date
    terms = list(zip(settle, maturity, file['coupon'], file['yield_pct']))
    rebuilt = pd.DataFrame({
        'price': [bondmath.price(*bond) for bond in terms],
        'accrued': [bondmath.accrued_interest(s, m, c) for s, m, c, y in terms],
        'yield_pct': [bondmath.yield_to_maturity(s, m, c, p) for s, m, c, p
                      in zip(settle, maturity, file['coupon'], file['price'])],
        'duration': [bondmath.modified_duration(*bond) for bond in terms],
    }, index=file['cusip'])
    return (rebuilt - file.set_index('cusip')[rebuilt.columns]).abs()
```

```
tolerance = pd.Series({'price': ROUNDED_3, 'accrued': ROUNDED_3,
                       'yield_pct': YIELD_FROM_ROUNDED_PRICE, 'duration': ROUNDED_3})
holdings_gaps = column_gaps(pd.read_csv('holdings.csv'))
assert (holdings_gaps <= tolerance).all().all()
print(len(holdings_gaps), 'holdings bonds, every gap inside its tolerance')
200 holdings bonds, every gap inside its tolerance
```

```
benchmark = pd.read_csv('benchmark.csv')
benchmark_gaps = column_gaps(benchmark)
outside = benchmark_gaps[(benchmark_gaps > tolerance).any(axis=1)]
print(len(benchmark_gaps), 'benchmark bonds,', len(outside), 'outside a tolerance')
outside.round(6)
821 benchmark bonds, 2 outside a tolerance
```

cusip	price	accrued	yield_pct	duration
99018SAA8	0.000162	0.028917	0.000019	0.005182
99128YAE4	0.004826	0.052333	0.000762	0.005623

Two bonds: the February month-end pair from notebook 06's Two Systems Disagree. Excel's month-end rule, which the module follows, puts their last coupon on 31 August; the data generator kept the 28th. Different coupon dates, different days accrued: "accrued", "duration" and one price differ.

```
month_end_pair = ['99018SAA8', '99128YAE4']
assert sorted(outside.index) == month_end_pair
others = benchmark_gaps.drop(month_end_pair)
assert (others <= tolerance).all().all()
print(len(others), 'other benchmark bonds, every gap inside its tolerance')
819 other benchmark bonds, every gap inside its tolerance
```

819 of 821 tie; the two that do not are notebook 06's February month-end bonds, a convention, not an error.

10.10 The book in dollars per basis point

Two totals describe the book's rate risk: book DV01, each position's dollars per bp added up (notebook 09), and the market-value weighted duration. Market value uses the dirty price: par held x (price + accrued) / 100.

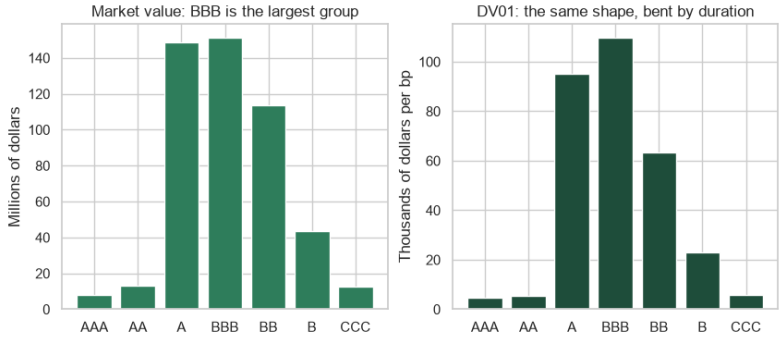
```
book['market_value_calc'] = book['par_held'] * (book['price'] + book['accrued']) / 100
total_market_value = book['market_value_calc'].sum()
assert abs(total_market_value - book['market_value'].sum()) < 0.005
print(f'Market value {total_market_value:,.2f} dollars')
Market value 490,819,215.00 dollars
```

```
book['dv01_dollars'] = [bondmath.dv01(*bond) * par / 100
                        for bond, par in zip(inputs, book['par_held'])]
book_dv01_dollars = book['dv01_dollars'].sum()
weights = book['market_value_calc'] / total_market_value
book_duration = (weights * book['duration_calc']).sum()
print(f'Book DV01 {book_dv01_dollars:,.2f} dollars per bp, duration {book_duration:.4f} years')
```

Job	Excel	Result
Book DV01	=SUMPRODUCT(E2:E201,H2:H201)/100	306,996.11
Market value	=SUM(I2:I201)	490,819,215.00
Book duration	=SUMPRODUCT(I2:I201,J2:J201)/SUM(I2:I201)	

AA carries 414 dollars of DV01 per million of market value and BBB 725: where duration differs, the shapes differ. This describes the book; it judges no group.

rating_group	market_value_calc	dv01_dollars	dv01_per_million
AAA	8200605.0	4679.0	571.0
AA	13048305.0	5403.0	414.0
A	148505040.0	95015.0	640.0
BBB	151163415.0	109669.0	725.0
BB	113754240.0	63537.0	559.0
B	43595535.0	23096.0	530.0
CCC	12552075.0	5598.0	446.0



The checks behind every answer, and bring it home

Check	Result
convexity against Excel's three PRICE cells	294 rows within 1e-3, Halvering 4.8e-06
pytest	28 passed
Price, accrued, duration against the file	200 of 200 within 0.0005 + 1e-9
Yield from the file's price	200 of 200 within 0.001 percent, largest 0.000440
Benchmark	819 of 821, two named month-end bonds
Market value	490,819,215.00 dollars, to the cent

You Can Now

Say where duration's line misses a full reprice, and why the miss grows with the square of the move.

Compute convexity with "bondmath.convexity" and say how other sources scale it.

Check convexity against three "PRICE" cells, since Excel has no convexity function.

Estimate a price change with duration and convexity.

Rebuild a file's four columns at stated tolerances, and leave out a known exception by name.

Compare two tags with "git diff --stat" and "git log".

Copy the work folder's two files into your "my_bondmath" folder; the copy replaces yours, so anything of your own must be in the work folder first. In PowerShell in VS Code, with the course's ".venv" active and the folder name the setup printed:

```
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$env:TEMP\pfi_course2_10_k4w7q2xr\bondmath.py", "$env:TEMP\pfi_course2_10_k4w7q2xr\test_bondmath.py" .
python -m pytest -q
git diff --stat
git commit -am "Add convexity, checked against a central difference on Excel PRICE"
git tag week2
git diff --stat week1 week2
```

pytest shows 28 passed, more with your exercise tests. In Colab, download the two files from the work folder under "/tmp".

Predict first: the price after 100 bp, and convexity

Predict 1

Duration's line says Halvering loses 6.7511 per 100 if its yield rises 100 bp. The curve bends. What does the full reprice lose, per 100? Write down a number.

```
moves_bp = list(range(-300, 301, 100))
misses = pd.DataFrame({'move_bp': moves_bp})
misses['full'] = [bondmath.dirty_price(settlement, halvering_maturity, 7.5, 7.7 + bp / 100)
                  - dirty_now for bp in moves_bp]
misses['line'] = [-modified * bp / 10000 * dirty_now for bp in moves_bp]
misses['miss'] = misses['full'] - misses['line']
misses.round(4)
```

Predict 2

Halvering's modified duration, 6.76 years, is 2.6 times Pellwick's 2.61. Its convexity is:

A) about 2.6 times Pellwick's

B) about 7 times

C) about 26 times

```
importlib.reload(bondmath)
bonds['modified'] = [bondmath.modified_duration(settlement, m, c, y)
                     for m, c, y in zip(maturities, coupons, yields)]
bonds['convexity'] = [bondmath.convexity(settlement, m, c, y)
                      for m, c, y in zip(maturities, coupons, yields)]
bonds[['issuer', 'modified', 'convexity']].round(4)
```

Predict first: two tags, and where yields miss

Predict 3

Today's commit added 30 and 9 lines; "week1" is five commits back. "git diff --stat week1 week2" shows:

A) every line added since "week1", notebooks 06 to 10

B) only today's 30 and 9

C) nothing, since a tag is only a name

```
!git -C "{work}" diff --stat week1 week2
!git -C "{work}" log --oneline week1..week2
```

Predict 4

Every price in the file carries up to 0.0005 of rounding. Solved back into a yield, which bonds miss the file's yield most?

A) the longest

B) no pattern

C) the shortest

```
book['yield_calc'] = [bondmath.yield_to_maturity(s, m, c, p) for s, m, c, p
                      in zip(book['settle'], book['maturity_date'], book['coupon'], book['price'])]
book['duration_calc'] = [bondmath.modified_duration(*bond) for bond in inputs]
book['yield_gap'] = (book['yield_calc'] - book['yield_pct']).abs()
assert book['yield_gap'].max() <= YIELD_FROM_ROUNDED_PRICE
book.nlargest(5, 'yield_gap')[['cusip', 'duration_calc', 'price', 'yield_gap']].round(6)
```

Predict first: two routes to one number

Predict 5

Book duration x market value x 0.0001 is a second route to book DV01, using the file's rounded prices. Against the summed DV01 column, it is:

A) equal to the cent

B) under 1 dollar apart

C) over 100 dollars apart

```
duration_route = book_duration * total_market_value * 0.0001
print(f'Duration route {duration_route:,.2f}, summed DV01 {book_dv01_dollars:,.2f} dollars')
print(f'Difference {book_dv01_dollars - duration_route:,.2f} dollars')
```

The note, answered

The note asked	Answer
The line's miss at 100 bp	Halvering 0.2879 per 100 up, 0.3073 down
The bend in bondmath	convexity, Halvering 59.5474 years squared, Excel's cells within 4.8e-06
The file's four columns	200 of 200 bonds within rounding
The benchmark	819 of 821, two February month-end bonds by convention
The book	306,996.11 dollars per bp, duration 6.2548 years

Be proud: you rebuilt someone else's book from your own module and can say where every number comes from.

Next: Treasury Curve Interpolation (notebook 11) splits each yield into a curve and a spread.